

Supplementary Material - HOFEEES: Higher-Order Automated Feature Engineering through Exhaustive Search of Low Dimensional Feature Subspaces

Anonymous submission

This supplementary material provides additional details of the proposed methodology that were omitted from the main paper due to space limitations. Specifically, it presents the complete AutoFE framework, including data preprocessing, feature generation, feature selection and the operators used within the AFE method. In addition, it includes algorithmic pseudocode to facilitate reproducibility and provide a deeper understanding of the proposed approach.

S1 Detailed Methodology

S1.1 Notation

Let the training data be a tabular matrix $X \in \mathbb{R}^{n \times d_0}$ with n samples and d_0 input features, and target vector $y \in \mathbb{R}^n$ (regression) or $y \in \{1, \dots, C\}^n$ (classification). HOFEEES constructs a sequence of engineered feature matrices $\{X^{(1)}, X^{(2)}, \dots, X^{(L)}\}$, where L denotes the number of stages for which the method runs. Each stage applies: (i) a per-feature unary transformation, (ii) binary operators for each of the possible pairs of features available at a given stage to generate new features (interaction mining), and (iii) a selection mechanism that retains a subset of the generated features for the next stage. The sequence of feature matrices $\{X^{(1)}, X^{(2)}, \dots, X^{(L)}\}$ corresponds to the successive feature engineering (FE) stages illustrated in Fig. 1 of the main paper.

Other AFE systems also follow a stage-wise process in which simple transformations are first applied to the original features, and the resulting features are iteratively reused in subsequent stages, enabling the gradual construction of higher-order feature interactions (Lin et al. 2024; Eldeeb, Amashukeli, and ElShawi 2022; Eldeeb and Elshawi 2024; Roshanski, Kalech, and Rokach 2023; Overman, Klabjan, and Utke 2024; Zhang et al. 2023). However, many of these methods rely on guided search strategies that explore only selected regions of the transformation space. In contrast, exhaustive approaches that attempt to systematically construct higher-order feature interactions often suffer from severe scalability limitations, as will be discussed in the later sections. In contrast, our proposed approach applies local,

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

pairwise exhaustive search to a set of features available at a given stage before a selection mechanism is applied to effectively manage the size of the feature space available at the following feature transformation stage. The applied selection criterion is of crucial importance and will be described in more detail in the following sections.

S1.2 Safe operator library

HOFEEES uses a library of numerically safe unary and binary operators.

Unary operators For each input feature vector $x \in \mathbb{R}^n$, the safe variants clip invalid domains (e.g., $\log(1+x)$ requires $x > -1$) and avoid numerical instabilities (e.g., reciprocal near zero). Table S1 provides a detailed mathematical description of each unary operator used in the method, including their functional form and the safety constraints applied to ensure numerical stability.

Table S1: Unary operators used in HOFEEES and their mathematical definitions.

Operator	Mathematical definition
Identity	$u(x) = x$
Absolute value	$u(x) = x $
Square	$u(x) = x^2$
Cube	$u(x) = x^3$
Square root	$u(x) = \sqrt{\max(x, 0)}$
Logarithm (log1p)	$u(x) = \log(1 + \max(x, -0.999999))$
Reciprocal	$u(x) = \begin{cases} \frac{1}{x}, & x > 10^{-6} \\ 0, & x \leq 10^{-6} \end{cases}$

Binary operators Table S2 provides the mathematical definitions of the binary operators used for interaction feature construction, including the safety mechanisms applied to ensure numerical stability.

The operator set used in HOFEEES is composed of generic transformations that are already well-established in the AFE literature. Similar unary (e.g., log, square, square root) and

Table S2: Binary operators used in HOFEEs for constructing interaction features. All operators are applied element-wise to pairs of input features, with safe variants used where necessary to prevent numerical instability.

Operator	Mathematical definition
Addition	$b(a, b) = a + b$
Subtraction	$b(a, b) = a - b$
Multiplication	$b(a, b) = a \cdot b$
Division	$b(a, b) = \begin{cases} \frac{a}{b}, & b > 10^{-6} \\ 0, & b \leq 10^{-6} \end{cases}$
Absolute difference	$b(a, b) = a - b $
Minimum	$b(a, b) = \min(a, b)$
Maximum	$b(a, b) = \max(a, b)$
Norm ratio	$b(a, b) = \frac{a}{ a + b + 10^{-6}}$

binary (sum, difference, product) operators have been extensively used in prior systems (Khurana et al. 2016; Eldeeb and Elshaw 2024; Eldeeb, Amashukeli, and Elshaw 2022; Horn, Pack, and Rieger 2020; Katz, Shin, and Song 2016; YILMAZ and ATASEVER 2025). As such, the operator library does not introduce methodological novelty, and differences between HOFEEs and existing approaches arise from other components of the framework rather than the choice of transformations.

S1.3 Statistical dependence measurement between a candidate feature and the target

Each candidate engineered feature is assessed with regard to its correlation to the target. Specifically, distance correlation is used in assessing candidate features after application of the unary operators to the available input features for both regression and classification, while candidate features resulting from the application of the binary operators to pairs of available features are assessed using Gaussian-copula based mutual information for regression and the method proposed in Ross for classification. These measures are referred to as “dependency scores” in the remainder of the document. We use these dependency scores to identify the most suitable unary transformation for each feature and the most suitable binary transformation for each feature pair. Unlike many existing AFE systems that retain engineered features primarily based on downstream ML model performance feedback (Overman, Klabjan, and Utke 2024; Horn, Pack, and Rieger 2020), the proposed approach evaluates transformations directly through their statistical relationship with the target variable. This reduces the risk of the FE process becoming overly biased toward the inductive biases or preferences of a specific learning algorithm. Furthermore, evaluating candidate transformations directly through their statistical relationship with the target enables the method to explore comparatively larger transformation spaces more efficiently than in the case of wrapper based methods. The description of the characteristics and the reasons for the selection of these particular feature assessment measures follows.

Distance correlation Distance correlation is a statistical measure used to quantify the dependence between two sets of variables. It equals zero if and only if the variables are independent and can detect both linear and non-linear relationships. Distance correlation has been previously utilised in AFE literature, primarily for identifying relationships between features rather than directly guiding feature transformation. For instance, Kaul, Maheshwary, and Pudi employed distance correlation to detect both linear and non-linear relationships between feature pairs prior to feature construction, after which regression models are fitted to generate new features. Similarly, Wang, Ding, and Pan computes distance correlation (for feature pairs) within each class and uses variations across classes to identify discriminative feature pairs, which are then used to construct new features via class-specific regression models. In these approaches, distance correlation serves as a mechanism for relationship discovery between independent variables.

In contrast, this study utilises distance correlation in a fundamentally different manner. Specifically, as described in Section S1.5, distance correlation is used to directly evaluate and select the most informative unary transformation for each feature with respect to the target variable. To the best of our knowledge, this is the first AFE method to employ distance correlation explicitly for identifying the most suitable unary transformation, thereby extending its role from relationship detection between independent variables, to direct optimisation of the AFE process. This is particularly advantageous as distance correlation can capture both linear and non-linear relationships between a feature transformation and the target, enabling the selection of the most suitable representation irrespective of the underlying functional form.

The use of distance correlation in this manner corresponds to the unary transformation components illustrated in Stages 1 to L of Fig. 1 of the main paper, where it is used to identify the most informative representation of each feature before interaction mining is performed.

Gaussian Copula Mutual Information A secondary regression scoring route evaluates the statistical relationship between a feature and the target via *Gaussian copula mutual information* (GCMI). The method proceeds by first mapping each variable to an approximately standard normal distribution using a rank-based transformation, followed by applying the closed-form Gaussian mutual information formula. GCMI is primarily introduced in this study (rather than using the distance correlation) to improve the computational efficiency during large-scale binary feature evaluation, where the number of candidate interactions grows combinatorially with the number of features and operators ($\binom{d}{2} \times |\mathcal{O}|$, where d denotes the number of features and $|\mathcal{O}|$ denotes the number of binary operators). In contrast, the unary transformation stage involves a substantially smaller search space ($d \times |\mathcal{O}|$ evaluations). The use of computationally efficient statistical measures enables the method to explore comparatively large transformation spaces while maintaining feasible computation time. This dependency score is used to identify the most suitable transformation for combining each pair of features, with further details provided in Section S1.5.

Rank-Based Inverse Normal Transform (INT) Given a variable $x = (x_1, \dots, x_n)$ with n samples, each observation is replaced by its empirical rank $r_i \in \{1, \dots, n\}$. A rank-based inverse normal transformation (INT) is then applied to obtain a variable which follows a Gaussian distribution.

$$x_i^* = \Phi^{-1}\left(\frac{r_i - c}{n - 2c + 1}\right) \quad (\text{S1})$$

where $\Phi^{-1}(\cdot)$ is the standard normal quantile function and c is an offset constant. In this work, we adopt $c = 0.5$, known as the *Rankit* choice (Beasley, Erickson, and Allison 2009), which yields

$$x_i^* = \Phi^{-1}\left(\frac{r_i - 0.5}{n}\right) \quad (\text{S2})$$

This transformation is monotonic in the original variable and produces marginal distributions that are approximately $\mathcal{N}(0, 1)$.

Relation to the Error Function The standard normal quantile function may be expressed in terms of the inverse error function:

$$\Phi^{-1}(p) = \sqrt{2} \operatorname{erf}^{-1}(2p - 1), \quad p \in (0, 1) \quad (\text{S3})$$

To compute erf^{-1} efficiently, we use a numerical approximation based on Winitzki’s closed-form formula (Winitzki 2008):

$$\begin{aligned} \operatorname{erf}^{-1}(m) \approx \operatorname{sign}(m) & \left[\left(\frac{2}{\pi a} + \frac{1}{2} \ln(1 - m^2) \right)^2 \right. \\ & \left. - \frac{1}{a} \ln(1 - m^2) \right]^{1/4} \\ & - \operatorname{sign}(m) \left(\frac{2}{\pi a} + \frac{1}{2} \ln(1 - m^2) \right)^{1/2} \end{aligned} \quad (\text{S4})$$

where $a = 0.147$.

Closed-Form Gaussian Mutual Information After applying the INT to a feature x and target y , yielding variables which follow a Gaussian distribution Z_x and Z_y , the relationship is estimated under a Gaussian copula model. Let Σ_x and Σ_y denote the covariance matrices of Z_x and Z_y , and let Σ denote the joint covariance of the concatenated vector $[Z_x, Z_y]$. The GCMI is then

$$I(x; y) = \frac{1}{2} \log \left(\frac{|\Sigma_x| |\Sigma_y|}{|\Sigma|} \right), \quad (\text{S5})$$

where $|\cdot|$ is the determinant.

This estimator is computationally efficient, making it suitable for repeated scoring within an AFE pipeline. However, it implicitly assumes that the variables which follow a Gaussian distribution, also follow a jointly multivariate Gaussian distribution. Although the rank-based transformation guarantees that each marginal distribution is approximately standard normal, it does not ensure joint normality. Hence, the Gaussian copula assumption may be violated in practice.

To the best of our knowledge, GCMI has not previously been employed within the AFE literature as a scoring mechanism for evaluating candidate feature transformations. In the proposed framework, it is used to efficiently assess binary interaction candidates during interaction mining, thereby enabling the exploration of substantially larger transformation spaces while maintaining computational tractability.

Numerical Stabilisation In practice, the covariance matrices Σ_x , Σ_y , or the joint covariance Σ may become singular or ill-conditioned due to collinearity, limited sample size, or numerical rounding effects. To ensure stable evaluation of the log-determinants in Eq. (S5), a small ridge term is added to the diagonal of any covariance matrix where the determinant is non-positive. Specifically, when $\det(\Sigma_x) \leq 0$, $\det(\Sigma_y) \leq 0$, or $\det(\Sigma) \leq 0$, the adjusted matrices

$$\Sigma_x \leftarrow \Sigma_x + 10^{-8} I, \quad \Sigma_y \leftarrow \Sigma_y + 10^{-8} I, \quad \Sigma \leftarrow \Sigma + 10^{-8} I$$

are used, where I denotes the identity matrix of appropriate dimension. This regularisation ensures positive definiteness and enables numerically stable computation of the GCMI.

Mutual information for classification In the case of classification, feature–target relationship for binary interaction candidates is quantified using mutual information as implemented in the scikit-learn `mutual_info_classif` estimator. This estimator computes a non-parametric estimate of mutual information between each candidate feature and the class labels, with optional handling of discrete-valued features for categorical inputs. This estimator implements the approach proposed by Ross.

Unlike many existing AFE methods where mutual information is applied as a global feature selection step after a large pool of candidate features has been generated (YILMAZ and ATASEVER 2025; Wang, Ding, and Pan 2020), the proposed approach employs mutual information locally during binary interaction discovery to evaluate competing transformations for each feature pair. Candidate interaction features are first constructed, after which mutual information is used to retain only the most informative transformation for a given feature pair. This allows the search space to be pruned early while preserving transformations that exhibit strong linear or non-linear relationships with the target variable. This early pruning strategy substantially reduces the number of generated features that must be retained, thereby mitigating memory requirements when operating on high-dimensional datasets.

The use of mutual information in this manner corresponds to the binary interaction mining components illustrated in Stages 1 to L of Fig. 1 of the main paper, where it is used to identify the most informative transformation for each candidate feature pair.

S1.4 Categorical variable encoding

Categorical variables are encoded using a frequency-based encoding scheme to obtain a purely numeric feature representation compatible with the downstream dependency scores. The encoding is designed to be simple and leakage-safe. The use of this encoding strategy corresponds to the categorical

Algorithm 1: Encoding and Preprocessing

Require: Raw feature matrix X

Ensure: Numeric preprocessed matrix $\tilde{X}$ and feature-type mapping τ

```
1:  $\tilde{X} \leftarrow \emptyset$ 
2:  $\tau \leftarrow \emptyset$ 
3: for each feature column  $x_j \in X$  do
4:   if all non-missing values in  $x_j$  can be converted to numeric values then
5:     Treat  $x_j$  as numerical
6:     Compute median value  $m_j$  from the training data
7:     Replace missing values in  $x_j$  with  $m_j$ 
8:     Convert  $x_j$  to floating point values
9:     Append processed  $x_j$  to  $\tilde{X}$ 
10:    Set  $\tau(x_j) \leftarrow$  numeric
11:   else
12:     Treat  $x_j$  as categorical
13:     Compute modal category  $c_j^*$  from the training data
14:     Replace missing values in  $x_j$  with  $c_j^*$ 
15:     for each observed category  $c$  in  $x_j$  do
16:       Compute relative frequency:
17:        $p(c) \leftarrow \frac{\#(x_j=c)}{n}$ 
18:     end for
19:     for each sample value  $x_{ij}$  in  $x_j$  do
20:       if  $x_{ij}$  was observed during training then
21:         Replace  $x_{ij}$  with  $p(x_{ij})$ 
22:       else
23:         Replace  $x_{ij}$  with 0
24:       end if
25:     end for
26:     Append frequency-encoded  $x_j$  to  $\tilde{X}$ 
27:     Set  $\tau(x_j) \leftarrow$  categorical
28:   end if
29: end for
30: return  $\tilde{X}, \tau$ 
```

encoding component illustrated in Fig. 1 of the main paper, where raw input features are transformed into a consistent numerical representation before entering the multi-stage FE pipeline. A detailed description of the encoding process is provided in Algorithm 1.

Feature type inference During fitting, each input column is assigned a feature type (numeric or categorical). The columns are inferred automatically; if all non-missing values can be safely converted to numeric values (without producing any NaNs), the column is treated as numeric. Otherwise, it is treated as categorical.

Handling of missing values Missing values are handled differently depending on feature type. For numeric features, missing values are imputed using the median computed from the training data. For categorical features, missing values are imputed using the modal category (most frequent value). These imputations are learned during fitting and reused unchanged during transformation.

Frequency encoding of categorical variables Categorical variables are encoded using relative frequency encoding. For a categorical feature x with observed categories $\{c_1, \dots, c_K\}$, each category is mapped to its empirical fre-

quency in the training data:

$$\text{enc}(c_k) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(x_i = c_k) \quad (\text{S6})$$

where $\mathbb{I}(\cdot)$ is the indicator function. This produces a single continuous-valued representation per categorical feature, avoiding the dimensionality expansion associated with one-hot encoding.

Handling numeric-like categorical variables In cases where categorical variables contain values that are inherently numeric (e.g., ordinal levels such as $\{1, 2, 3, \dots\}$), no encoding is applied and the values are retained in their original form. Formally, if a feature x takes values in $\mathbb{R}$ and preserves a meaningful numerical order, the encoding function is defined as:

$$\text{enc}(x_i) = x_i \quad (\text{S7})$$

This avoids unnecessary transformation and ensures that the inherent ordinal relationships are preserved for downstream processing.

Handling of unseen categories During transformation, categories not observed during fitting are encoded using a fixed fallback value. In this work, unseen categories are assigned a frequency value of 0, ensuring deterministic behaviour and preventing missing values from propagating into the feature matrix.

This encoding strategy ensures that all features entering the AFE pipeline are numeric and low-dimensional, while maintaining robustness to missing values and unseen categories.

Comparison with existing approaches Many existing AFE systems employ simple encoding strategies for categorical variables, such as ordinal encoding applied across the entire dataset (Eldeeb and Elshawi 2024; Eldeeb, Amashukeli, and ElShawi 2022). Such approaches can introduce unintended information leakage. Furthermore, applying numerical transformation operators (e.g., $\sqrt{\cdot}$ or logarithmic transformations) to such encoded variables can lead to semantically invalid feature constructions.

To avoid these issues, AFE methods such as AutoFeat (Horn, Pack, and Rieger 2020) adopt one-hot encoding and exclude categorical variables from the feature generation process. While this prevents invalid transformations, it can lead to a significant increase in dimensionality when high-cardinality features are present. Additionally, the one-hot encoding scheme in AutoFeat does not robustly handle categories that appear only in the test set, but not in the train set. That is, the method throws errors in such a scenario.

In contrast, the proposed encoding strategy is specifically designed to address these limitations by maintaining a compact representation, avoiding the application of arithmetic transformations to categorical variables, and ensuring consistent handling of previously unseen categories.

S1.5 One AFE stage: unary search + interaction mining

A single stage takes an input matrix $X^{(\ell-1)} \in \mathbb{R}^{n \times d_{\ell-1}}$ and produces an output matrix $X^{(\ell)}$ that concatenates (i) the se-

lected unary-transformed columns and (ii) selected binary interaction features (features constructed as a result of applying a binary operator to a pair of features). This corresponds to a single FE stage within the multi-stage pipeline illustrated in Fig. 1 of the main paper, which is repeatedly applied from Stage 1 to Stage L .

Unary layer: best transformation per feature with margin constraint Let $s(\cdot; \cdot)$ denote the feature–target dependency score, implemented using distance correlation for unary candidates. For each input column x_j , a unary transformation is selected by maximizing the dependency score with the target:

$$u_j^* = \arg \max_{u \in \mathcal{U}} s(u(x_j); y), \quad \tilde{x}_j = u_j^*(x_j) \quad (\text{S8})$$

Identity is always evaluated as a baseline, and a non-identity unary transformation is retained only if it exceeds the identity score by a margin ε :

$$s(\tilde{x}_j; y) > s(x_j; y) + \varepsilon \quad (\text{S9})$$

Otherwise, the identity transformation is selected, where the margin ε is a user-defined margin hyperparameter.

Categorical inputs are passed through the identity transformation, as applying continuous mathematical operators to frequency-encoded categorical variables is not semantically meaningful and may introduce artificial feature relationships.

The columns which have undergone unary transformations are stacked to form:

$$U = [\tilde{x}_1, \dots, \tilde{x}_{d_{\ell-1}}] \in \mathbb{R}^{n \times d} \quad (\text{S10})$$

where d is the original number of columns in this stage.

The complete unary transformation search procedure is presented in Algorithm 2.

Binary interaction mining: exhaustive search and acceptance rule For each unordered feature pair (i, j) (with $1 \leq i < j \leq d$), a set of candidate interaction features is constructed:

$$z_{ij}^{(b)} = b(U_{\cdot i}, U_{\cdot j}), \quad b \in \mathcal{B} \quad (\text{S11})$$

Each candidate interaction feature is scored against the target using mutual information, computed via Gaussian-copula-based mutual information for regression and via the scikit-learn `mutual_info_classif` estimator for classification. For each pair (i, j) , at most one binary operator is retained: the one yielding the highest score for that pair, subject to an acceptance criterion.

Operand-beating constraint with margin A binary candidate is kept only if its dependency score exceeds the score of *each* operand by a margin δ :

$$s(z_{ij}^{(b)}; y) > \max\{s(U_{\cdot i}; y), s(U_{\cdot j}; y)\} + \delta \quad (\text{S12})$$

Exclusion of categorical variables from pairwise feature construction Pairwise feature construction is restricted to numeric features. Interactions between categorical variables are not explicitly constructed, as the arithmetic operators considered in this work do not have meaningful semantics

Algorithm 2: Unary Transformation Search

Require: Input feature matrix $X^{(\ell-1)}$
Require: Target vector y
Require: Unary improvement margin ϵ
Ensure: Unary-transformed feature set $U^{(\ell)}$

```

1:  $U^{(\ell)} \leftarrow \emptyset$ 
2: for each feature  $x_j \in X^{(\ell-1)}$  do
3:   if  $x_j$  is categorical then
4:     Retain identity transformation only
5:      $\tilde{x}_j \leftarrow x_j$ 
6:     Append  $\tilde{x}_j$  to  $U^{(\ell)}$ 
7:     continue
8:   end if
9:   Apply identity transformation (Returns the original variable):
10:   $x_j^{\text{id}} \leftarrow x_j$ 
11:  Compute identity dependency score:
12:   $s_{\text{id}} \leftarrow D(x_j^{\text{id}}, y)$ 
13:   $s^* \leftarrow s_{\text{id}}$ 
14:   $u^* \leftarrow \text{identity}$ 
15:   $x_j^* \leftarrow x_j^{\text{id}}$ 
16:  for each non-identity unary operator  $u$  do
17:    Generate candidate transformed feature:
18:     $x_j^u \leftarrow u(x_j)$ 
19:    Compute dependency score:
20:     $s_u \leftarrow D(x_j^u, y)$ 
21:    if  $s_u > s^*$  then
22:       $s^* \leftarrow s_u$ 
23:       $u^* \leftarrow u$ 
24:       $x_j^* \leftarrow x_j^u$ 
25:    end if
26:  end for
27:  if  $u^* \neq \text{identity}$  and  $s^* > s_{\text{id}} + \epsilon$  then
28:     $\tilde{x}_j \leftarrow x_j^*$ 
29:  else
30:     $\tilde{x}_j \leftarrow x_j^{\text{id}}$ 
31:  end if
32:  Append  $\tilde{x}_j$  to  $U^{(\ell)}$ 
33: end for
34: return  $U^{(\ell)}$ 
```

when applied to frequency-encoded categorical representations and can lead to uninterpretable or spurious features. Let cat_1 and cat_2 denote categorical variables and A, B, X, Z denote category values. Rule-based categorical interactions, such as equality tests between categorical variables (e.g., $\text{cat}_1 = \text{cat}_2$), conjunctions across multiple categorical variables (e.g., $\text{cat}_1 = A \wedge \text{cat}_2 = B$), and more complex logical patterns, including disjunctions of conjunctions (e.g., $(\text{cat}_1 = A \wedge \text{cat}_2 = X) \vee (\text{cat}_1 = B \wedge \text{cat}_2 = Z)$), are meaningful constructions. However, these are naturally captured by tree-based learning algorithms through hierarchical split conditions, making explicit construction of categorical–categorical interaction features unnecessary.

Interactions between continuous and categorical variables can be meaningfully represented when categorical variables are one-hot encoded and combined via multiplication. However, one-hot encoding substantially increases feature dimensionality and can lead to feature space explosion. Since categorical variables are frequency-encoded in this work to main-

tain a compact and interpretable representation, arithmetic interactions between continuous features and frequency-encoded categories are excluded. This design choice is also consistent with other existing AFE systems, which similarly exclude explicit categorical feature construction (Horn, Pack, and Rieger 2020).

Growth constraint for higher interaction levels To promote increasing interaction order across stages, subsequent stages enforce that each candidate pair includes at least one feature that was newly created in the previous stage. Formally, let $m_j \in \{0, 1\}$ indicate whether feature j was newly generated in the previous stage. A candidate pair (i, j) is admissible only if at least one of its operands was newly generated:

$$m_i = 1 \text{ or } m_j = 1.$$

This constraint is applied from stage 2 onward in the multi-stage workflow. Similar growth constraints have also been adopted in prior AFE systems (Overman, Klabjan, and Utke 2024).

The complete binary transformation search procedure is presented in Algorithm 3.

Distinction from existing transformation selection strategies The feature transformation selection strategy employed in HOFEEES differs from prior AFE approaches in several important ways. Many existing systems rely on heuristic or guided search strategies that do not explore the full transformation space. In contrast, exhaustive AFE approaches generate and retain all possible transformations, resulting in a rapid growth in feature dimensionality that imposes significant computational and memory constraints, thus failing on high dimensional datasets (Horn, Pack, and Rieger 2020; Zhang et al. 2023), and preventing the exploration of higher order feature interactions. In contrast, HOFEEES retains at most one transformation per feature or feature pair, and only when the constructed feature has a greater relationship with the target, allowing the method to explore the transformation space exhaustively while controlling feature space growth and avoiding the severe memory explosion typically associated with exhaustive AFE methods on high-dimensional datasets.

Although HOFEEES shares certain structural similarities with the approach of Overman, Klabjan, and Utke, such as retaining at most one transformation per feature pair, the underlying feature selection strategy is fundamentally different. Overman, Klabjan, and Utke pre-filters candidate pairs using interaction information prior to feature construction, thereby discarding feature pairs that may exhibit limited pairwise relationship with the target but could still contribute useful information when combined with other variables in the feature space. In contrast, HOFEEES performs exhaustive interaction exploration and retains candidate features whenever they demonstrate stronger relationship with the target than their constituent operands. Furthermore, through the use of permutation-based importance (see Section S1.6), features with weak pairwise marginal relationship may still be preserved if they contribute useful predictive information jointly with other variables under the downstream model. Thus, HOFEEES enables exhaustive exploration of the interaction

space while avoiding the premature elimination of features that may initially exhibit weak marginal pairwise relevance but later become informative through higher-order interactions involving three or more variables.

This approach allows HOFEEES to perform an exhaustive exploration of the transformation space by evaluating all candidate unary and pairwise binary transformations at each stage, while being capable of exploring higher order feature interactions.

Furthermore, the use of margin-based acceptance criteria (ϵ and δ) also allows the method to effectively suppress unnecessary or harmful FE. In cases where the original feature space is already well-suited for the learning task (e.g., separable datasets such as Iris), additional transformations may degrade model performance by introducing redundant features. In HOFEEES, such transformations are automatically rejected if they fail to exceed the score of their original counterparts by the required margins. Consequently, the method can naturally converge to a no-transformation system (thus returning the original dataset) when FE is not beneficial. While several existing AFE systems similarly retain generated features only when they improve validation performance (Bouadi et al. 2025b), the novelty of HOFEEES lies in explicitly formalizing this retention process through principled margin-based acceptance criteria. That is, generated features are not retained solely based on marginal improvements in downstream model performance, but are rather required to demonstrate sufficiently stronger statistical relationships with the target variable.

S1.6 Multi-stage workflow and model-aware selection

Stage scheduling and growth enforcement The full workflow executes stage 1 with unrestricted pairwise feature construction. Stages 2 through L are then executed with a growth constraint enabled, requiring that at least one feature in each candidate pair corresponds to a feature newly generated in the immediately preceding stage. This creates a controlled progression toward higher-order interaction structure while preventing combinatorial blow-up. The number of stages L is a user-defined hyper-parameter that determines the maximum interaction depth explored by the method. This mechanism corresponds to the Stage 1 to L portion of Fig. 1 of the main paper.

Selection criteria After each stage produces its feature matrix, features are ranked. Two ranking strategies are supported: (i) distance correlation-based scoring with respect to the target, and (ii) permutation-based feature importance computed using a predictive model. Distance correlation-based scoring evaluates each feature independently and is effective for identifying strong marginal relationships with the target. However, it does not account for joint feature interactions and can fail in settings such as XOR-type problems, where individual features are uninformative in isolation. Permutation-based importance, when computed using interaction-capable models such as decision trees or ensembles, can capture such joint effects by measuring the impact of feature perturbations on model performance. Both ranking

modes are therefore provided. When distance correlation is used, an optional threshold may be applied to discard features whose marginal association with the target falls below a specified level. This threshold will be referred to as "dependency threshold" in the rest of the paper.

Model-aware feature selection and stage-wise selection modes After feature generation at each stage, the method optionally performs feature selection to control dimensionality and retain only informative representations. Three feature selection modes are supported:

- **Score-only selection:** In score-only selection, features are ranked using the distance correlation, and the top-ranked features are retained without fitting a downstream predictive model. In this mode, at the end of each stage, the feature set is truncated to retain the top n features, where n corresponds to the number of variables in the original dataset. This mode corresponds to the 'No Model Aware FS' branch illustrated in Fig. 2 of the main paper.
- **Model-aware selection applied at each stage:** Model-aware feature selection explicitly evaluates feature subsets using a predictive model and validation data, retaining the number of features that minimizes the validation error and thereby selecting a feature subset which is most appropriate for the downstream ML model. Feature selection is performed at the end of each stage. This mode corresponds to the 'Each Stage' branch illustrated in Fig. 2 of the main paper.
- **Model-aware selection applied only at the final stage:** Model-aware selection may be deferred until the final stage to prevent the premature discarding of useful feature combinations. In the latter case, intermediate stages apply distance-correlation-based ranking and retain a fixed number of features equal to the original input dimensionality, with model-aware selection applied only after the final stage. If preferred, permutation based importance may be used instead of distance correlation. This constraint is intended to control the growth of the feature space and computational cost while allowing newly generated features to compete directly with original features for inclusion in subsequent stages. This mode corresponds to the 'Final Stage' branch illustrated in Fig. 2 of the main paper.

Best- k selection via validation curves When model-aware feature selection is enabled, the ranked features are evaluated using a validation-based best- k procedure. A train/validation split is created, and a grid of candidate feature counts $\mathcal{K}$ is evaluated. For each $k \in \mathcal{K}$, the downstream model is trained on the top- k ranked features and evaluated on the validation set. The final model is then refitted on the full training data using the selected feature subset.

Formally, let π denote the ranking permutation such that features are ordered by decreasing score. For a given stage ℓ , define

$$\hat{f}_k^{(\ell)} = \mathcal{F}(X_{\text{train}, \pi(1:k)}^{(\ell)}, y_{\text{train}}) \quad (\text{S13})$$

where $\mathcal{F}$ denotes the learning algorithm that returns a fitted model $\hat{f}_k^{(\ell)}$ trained on the top- k features.

The validation score is then defined as

$$S(k) = \text{Score}\left(y_{\text{val}}, \hat{f}_k^{(\ell)}(X_{\text{val}, \pi(1:k)}^{(\ell)})\right), \quad k \in \mathcal{K} \quad (\text{S14})$$

where $\hat{f}_k^{(\ell)}(X_{\text{val}, \pi(1:k)}^{(\ell)})$ denotes the predictions on the validation set, and $\text{Score}(\cdot)$ is a task-specific evaluation metric (e.g., accuracy for classification or root mean squared error for regression).

The selected feature count is

$$k^* = \arg \max_{k \in \mathcal{K}} S(k). \quad (\text{S15})$$

The validation score ($\text{Score}(\cdot)$) is user-configurable. The candidate feature-count grid $\mathcal{K}$ is evaluated in fixed increments, up to a maximum budget, with both the step size and maximum number of evaluations treated as user-defined hyperparameters. This procedure corresponds to the feature selection workflow illustrated in Fig. 2 of the main paper. A detailed description of the model-aware feature selection procedure is provided in Algorithm 4.

Feature selection in prior AFE methods Existing AFE approaches employ a range of feature selection strategies to control the growth of the feature space. For instance, Eldeeb, Amashukeli, and ElShawi; Eldeeb and ElShawi uses importance scores derived from tree-based models, followed by Pearson correlation filtering to remove redundant features. Katz, Shin, and Song adopts a meta-learning-based ranking model to estimate the utility of candidate features, and subsequently applies a wrapper-based evaluation in which a feature is retained only if it improves the performance of a downstream model. Similarly, Kaul, Maheshwary, and Pudi employ stability selection via Randomized Lasso across subsamples, combined with a final Information Gain-based filtering step, while Chen et al. use Random Forest importance for initial feature filtering prior to transformation. Other approaches, such as Horn, Pack, and Rieger, combine correlation-based ranking with Lasso-based wrapper methods to iteratively select features that best explain the target.

To the best of our knowledge, however, this is the first AFE method to explicitly employ distance correlation for feature selection, thereby enabling the detection of general (including non-linear) relationships between features and the target. In contrast, commonly used measures such as Pearson correlation (Eldeeb and ElShawi 2024; Eldeeb, Amashukeli, and ElShawi 2022; Horn, Pack, and Rieger 2020) are limited to linear relationships, while model-specific importance measures (e.g., Random Forest importance) may introduce bias towards the inductive assumptions of a particular model and may not generalize well across different downstream learners (Katz, Shin, and Song 2016; Chen et al. 2019). Furthermore, the use of permutation-based importance allows the evaluation of feature relevance in the context of the full feature subspace, rather than in isolation.

Additionally, many existing AFE methods aim to construct a *generally optimal* feature space (Eldeeb, Amashukeli, and ElShawi 2022; Eldeeb and ElShawi 2024). However, this assumption is fundamentally flawed. The notion of a single optimal representation is inconsistent with the heterogeneous

nature of different ML models. For instance, instance-based learners such as k-nearest neighbours typically benefit from low-dimensional feature spaces, whereas ensemble methods like random forests can effectively exploit high-dimensional representations. Consequently, a feature space that is optimal for one model may be suboptimal for another. Existing AFE approaches either implicitly assume model-agnostic optimality or explicitly optimise for a single downstream learner (e.g., logistic regression) (Liu, Guo, and Xu 2024), limiting their general applicability. In contrast, the proposed method introduces a model-aware and metric-aware AFE framework that dynamically constructs feature spaces tailored to the specific downstream model and evaluation criterion specified by the user. While model-aware optimization has been explored in prior AFE systems (Katz, Shin, and Song 2016), HOFEEES extends this idea within a controlled exhaustive feature construction framework designed to support scalable exploration of higher-order feature interactions.

S1.7 Interpretability and Traceability in AFE

Many AFE systems focus on constructing interpretable feature spaces, aiming to balance predictive performance with transparency. Interpretability in this context is typically achieved through two key properties: semantic meaning and traceability. Semantic meaning refers to the ability of a feature to convey a clear and intuitive interpretation within the problem domain (e.g., extracting `month` from a date, deriving `country` from a `city`), while traceability ensures that each generated feature can be linked back to the original variables and transformations used in its construction (Eldeeb, Amashukeli, and ElShawi 2022; Zhang et al. 2025; Bouadi et al. 2025a).

In this work, interpretability is primarily aligned with the notion of traceability, whereby the construction of each feature can be understood directly from its representation. However, it is important to note that in multi-stage AFE systems, where features are iteratively constructed from previously generated features, the resulting feature expressions can become increasingly complex. Although such features remain technically traceable, their interpretability may degrade in practice, as the composition of multiple transformations across stages can obscure the underlying meaning of the feature.

The complete workflow of the proposed framework is summarized in Algorithm 5, which integrates the encoding, feature generation, interaction mining, and feature selection components described throughout this material.

Algorithm 3: Exhaustive Binary Interaction Mining

Require: Unary-transformed feature set $U^{(\ell)}$
Require: Target vector y
Require: Learning task T (Regression or Classification)
Require: Binary improvement margin δ
Require: Previously generated feature set $\mathcal{N}^{(\ell-1)}$
Require: Growth constraint flag G (enabled as True for Stages 2 to L , disabled for Stage 1)
Ensure: Retained binary interaction feature set $B^{(\ell)}$

```

1:  $B^{(\ell)} \leftarrow \emptyset$ 
2: for each unordered pair  $(u_i, u_j)$  in  $U^{(\ell)}$  do
3:   if  $G = \text{True}$  then
4:     if  $u_i \notin \mathcal{N}^{(\ell-1)}$  and  $u_j \notin \mathcal{N}^{(\ell-1)}$  then
5:       Discard pair  $(u_i, u_j)$ 
6:       continue
7:     end if
8:   end if
9:   if  $u_i$  or  $u_j$  is categorical then
10:     Discard pair  $(u_i, u_j)$ 
11:     continue
12:   end if
13:    $q^* \leftarrow -\infty$ 
14:    $b^* \leftarrow \text{None}$ 
15:    $z^* \leftarrow \text{None}$ 
16:   for each binary operator  $b$  do
17:     Generate candidate interaction:
18:      $z_{ij}^b \leftarrow b(u_i, u_j)$ 
19:     if  $T = \text{regression}$  then
20:       Compute interaction score:
21:        $q_b \leftarrow \text{GCMI}(z_{ij}^b, y)$ 
22:     else
23:       Compute interaction score:
24:        $q_b \leftarrow \text{MI}(z_{ij}^b, y)$ 
25:     end if
26:     if  $q_b > q^*$  then
27:        $q^* \leftarrow q_b$ 
28:        $b^* \leftarrow b$ 
29:        $z^* \leftarrow z_{ij}^{b^*}$ 
30:     end if
31:   end for
32:   if  $b^* = \text{None}$  then
33:     continue
34:   end if
35:   if  $T = \text{regression}$  then
36:      $q_i \leftarrow \text{GCMI}(u_i, y)$ 
37:      $q_j \leftarrow \text{GCMI}(u_j, y)$ 
38:   else
39:      $q_i \leftarrow \text{MI}(u_i, y)$ 
40:      $q_j \leftarrow \text{MI}(u_j, y)$ 
41:   end if
42:   if  $q^* > q_i + \delta$  and  $q^* > q_j + \delta$  then
43:     Retain  $z^*$  as a new interaction feature
44:     Append  $z^*$  to  $B^{(\ell)}$ 
45:   end if
46: end for
47: return  $B^{(\ell)}$ 

```

Algorithm 4: Model-Aware Feature Selection

Require: Candidate feature set $F^{(\ell)}$
Require: Target vector y
Require: Learning task T
Require: Learning algorithm M
Require: Validation metric $\mathcal{M}$
Require: Feature-selection mode $S \in \{\text{distance} - \text{correlation}, \text{permutation}\}$
Ensure: Selected feature set $X_{\text{sel}}^{(\ell)}$, newly generated selected feature set $N^{(\ell)}$

```

1: Remove features containing NaN, infinity, or numerically unstable values
2: if  $S = \text{distance} - \text{correlation}$  then
3:   for each feature  $f_j \in F^{(\ell)}$  do
4:     Compute distance correlation score:
5:      $r_j \leftarrow D(f_j, y)$ 
6:   end for
7: else if  $S = \text{permutation}$  then
8:   Fit model  $M$  on  $F^{(\ell)}$  and  $y$ 
9:   for each feature  $f_j \in F^{(\ell)}$  do
10:     Calculate and set permutation importance score  $r_j$ 
11:   end for
12: end if
13: Rank all features in descending order of  $r_j$ 
14: Split data into sub-training set and validation set
15: Define candidate feature counts:
16:  $\mathcal{K} \leftarrow \{k_{\text{step}}, 2k_{\text{step}}, \dots, rk_{\text{step}}\}$ 
17:  $m^* \leftarrow -\infty$ 
18:  $k^* \leftarrow 0$ 
19: for each  $k \in \mathcal{K}$  do
20:   Select the top- $k$  ranked features:
21:    $F_k \leftarrow \text{TopK}(F^{(\ell)}, k)$ 
22:   Train model  $M_k$  on the sub-training set using  $F_k$ 
23:   Evaluate  $M_k$  on the validation set:
24:    $m_k \leftarrow \mathcal{M}(M_k)$ 
25:   if  $m_k > m^*$  then
26:      $m^* \leftarrow m_k$ 
27:      $k^* \leftarrow k$ 
28:   end if
29: end for
30: Select final feature subset:
31:  $X_{\text{sel}}^{(\ell)} \leftarrow \text{TopK}(F^{(\ell)}, k^*)$ 
32: return  $X_{\text{sel}}^{(\ell)}$ , newly generated selected feature set  $N^{(\ell)}$ 

```

Algorithm 5: Overall HOFEEES Workflow

Require: Training feature matrix $X \in \mathbb{R}^{n \times d_0}$
Require: Target vector y
Require: Learning task $T \in \{\text{classification, regression}\}$
Require: Number of interaction stages L
Require: Unary improvement margin ϵ
Require: Binary improvement margin δ
Require: Downstream learning algorithm M
Require: Validation metric $\mathcal{M}$
Require: Feature-selection mode $S \in \{\text{distancecorrelation, permutation}\}$
Ensure: Final engineered feature matrix $X_{\text{sel}}^{(L)}$ and fitted model M^*
1: $X^{(0)} \leftarrow \text{ENCODEANDPREPROCESS}(X)$
2: $\mathcal{N}^{(0)} \leftarrow \emptyset \triangleright$ Set of newly generated features from the previous stage
3: **for** $\ell = 1$ to L **do**
4: $U^{(\ell)} \leftarrow \text{UNARYTRANSFORMATIONSEARCH}(X^{(\ell-1)}, y, \epsilon)$
5: **if** $\ell = 1$ **then**
6: $B^{(\ell)} \leftarrow \text{BINARYINTERACTIONMIN-}$
 $\text{ING}(U^{(\ell)}, y, T, \delta, \emptyset, \text{False})$
7: **else**
8: $B^{(\ell)} \leftarrow \text{BINARYINTERACTIONMIN-}$
 $\text{ING}(U^{(\ell)}, y, T, \delta, \mathcal{N}^{(\ell-1)}, \text{True})$
9: **end if**
10: $F^{(\ell)} \leftarrow U^{(\ell)} \cup B^{(\ell)}$
11: $(X_{\text{sel}}^{(\ell)}, \mathcal{N}^{(\ell)}) \leftarrow \text{MODEL-AWARE-FEATURE-SELEC-}$
 $\text{TION}(F^{(\ell)}, y, T, M, \mathcal{M}, S)$
12: $X^{(\ell)} \leftarrow X_{\text{sel}}^{(\ell)}$
13: **end for**
14: **return** $X_{\text{sel}}^{(L)}$

References

- Beasley, T. M.; Erickson, S.; and Allison, D. B. 2009. Rank-Based Inverse Normal Transformations are Increasingly Used, But are They Merited? *Behavior Genetics*, 39(5): 580–595.
- Bouadi, M.; Alavi, A.; Benbernou, S.; and Ouziri, M. 2025a. KnowFE: A Hybrid AI System for Explainable Feature Engineering using Knowledge-Guided Reinforcement Learning. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM '25*, 6609–6613. ACM.
- Bouadi, M.; Alavi, A.; Benbernou, S.; and Ouziri, M. 2025b. Synergizing Large Language Models and Knowledge-Based Reasoning for Interpretable Feature Engineering. In *Proceedings of the ACM on Web Conference 2025, WWW '25*, 2606–2620. ACM.
- Chen, X.; Qiao, B.; Zhang, W.; Wu, W.; Chintalapati, M.; Zhang, D.; Lin, Q.; Luo, C.; Li, X.; Zhang, H.; Xu, Y.; Dang, Y.; Sui, K.; and Zhang, X. 2019. Neural Feature Search: A Neural Architecture for Automated Feature Engineering. In *2019 IEEE International Conference on Data Mining (ICDM)*, 71–80. IEEE.
- Eldeeb, H.; Amashukeli, S.; and ElShawi, R. 2022. BigFeat: Scalable and Interpretable Automated Feature Engineering Framework. In *2022 IEEE International Conference on Big Data (Big Data)*. IEEE.
- Eldeeb, H.; and Elshawi, R. 2024. Empowering Machine Learning with Scalable Feature Engineering and Interpretable AutoML. *IEEE Transactions on Artificial Intelligence*, 1–16.
- Horn, F.; Pack, R.; and Rieger, M. 2020. *The autofeat Python Library for Automated Feature Engineering and Selection*, 111–120. Springer International Publishing. ISBN 9783030438234.
- Katz, G.; Shin, E. C. R.; and Song, D. 2016. ExploreKit: Automatic Feature Generation and Selection. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE.
- Kaul, A.; Maheshwary, S.; and Pudi, V. 2017. AutoLearn — Automated Feature Generation and Selection. In *2017 IEEE International Conference on Data Mining (ICDM)*, 217–226. IEEE.
- Khurana, U.; Turaga, D.; Samulowitz, H.; and Parthasarathy, S. 2016. Cognito: Automated Feature Engineering for Supervised Learning. In *2016 IEEE 16th International Conference on Data Mining Workshops (ICDMW)*. IEEE.
- Lin, Y.; Ding, B.; Jagadish, H.; and Zhou, J. 2024. SMART-FEAT: Efficient Feature Construction through Feature-Level Foundation Model Interactions.
- Liu, M.; Guo, C.; and Xu, L. 2024. An interpretable automated feature engineering framework for improving logistic regression. *Applied Soft Computing*, 153: 111269.
- Overman, T.; Klabjan, D.; and Utke, J. 2024. IIFE: Interaction Information Based Automated Feature Engineering. In *2024 IEEE International Conference on Data Mining (ICDM)*, 803–808. IEEE.
- Roshanski, I.; Kalech, M.; and Rokach, L. 2023. Automatic Feature Engineering for Learning Compact Decision Trees. *Expert Systems with Applications*, 229: 120470.
- Ross, B. C. 2014. Mutual Information between Discrete and Continuous Data Sets. *PLoS ONE*, 9(2): e87357.
- Wang, M.; Ding, Z.; and Pan, M. 2020. LbR: A New Regression Architecture for Automated Feature Engineering. In *2020 International Conference on Data Mining Workshops (ICDMW)*, 432–439. IEEE.
- Winitzki, S. 2008. A handy approximation for the error function and its inverse. *A lecture note obtained through private communication*.
- YILMAZ, M.; and ATASEVER, U. H. 2025. BSD-FE: a fast and effective framework for automated feature engineering. *Machine Learning: Science and Technology*, 6(4): 045075.
- Zhang, T.; Zhang, Z.; Fan, Z.; Luo, H.; Liu, F.; Liu, Q.; Cao, W.; and Li, J. 2023. OpenFE: automated feature generation with expert-level performance. In *Proceedings of the 40th International Conference on Machine Learning, ICML'23*. JMLR.org.
- Zhang, X.; Zhang, J.; Rekabdar, B.; Zhou, Y.; Wang, P.; and Liu, K. 2025. Dynamic and Adaptive Feature Generation with LLM. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-2025*, 7029–7037. International Joint Conferences on Artificial Intelligence Organization.